

SGC NAVIGATOR

Projektna dokumentacija

Splošni geslovník COBISS - Spletni navigator tezavra

Pavel Bozhinov, Dragan Stojanović - 2025



Fakulteta za elektrotehniko,
računalništvo in informatiko

Vsebina dokumenta

Vsebina dokumenta.....	2
1. Povzetek in opis rešitve.....	4
1.1 Namen in kontekst projekta.....	4
1.2 Opisana problematika in rešitev.....	4
1.3 Ciljni uporabniki.....	4
1.4 Primerjava s sorodnimi rešitvami.....	5
2. Arhitektura sistema.....	6
2.1 Pregled celotne arhitekture.....	6
2.2 Odjemalska stran (Frontend).....	6
2.3 Zaledna stran (Backend).....	7
2.4 Podatkovna baza Neo4j in SKOS podatkovni model.....	7
2.5 GraphQL API shema.....	8
2.6 Varnostna arhitektura - ALTCHA CAPTCHA.....	9
3. Struktura uporabniških vmesnikov.....	10
3.1 Vstopna stran - SearchPage.....	10
3.2 Stran koncepta - GraphPage.....	10
3.3 Navigacija in URL parametri.....	11
4. Zunanje odvisnosti aplikacije.....	12
4.1 Frontend - odvisnosti.....	12
4.2 Backend - odvisnosti.....	13
5. Izseki pomembne kode.....	15
5.1 GraphQL shema - schema.graphqls.....	15
5.2 Podatkovni model - SkosConcept.java.....	15
5.3 Projekcijski vmesnik - ConceptProjection.java.....	16
5.4 DTO za robove grafa - ConceptEdge.java.....	16
5.5 Cypher poizvedbe v ConceptRepository.java.....	17
5.6 ConceptService - logika iskanja po jeziku.....	17
5.7 ConceptGraphQLController - field-level resolverji.....	18
5.8 GatewayController - REST API za CAPTCHA.....	18
5.9 AltchaService - generiranje in preverjanje CAPTCHA.....	19
5.10 WebConfig - CORS konfiguracija.....	19
5.11 Vite konfiguracija - proxy za razvoj.....	20
5.12 RateLimitContext - odjemalska zaščita.....	20
5.13 Frontend GraphQL poizvedbe - GraphPage.tsx.....	21
5.14 Gradnja grafnih podatkov - useMemo v GraphPage.tsx.....	21
6. Zagon rešitve.....	23
6.1 Predpogoji.....	23
6.2 Kloniranje repozitorijev.....	23
6.3 Zagon zalednega sistema (Backend).....	23
6.4 Zagon čelnega sistema (Frontend).....	24
6.5 Zagon celotnega sklopa z Docker Compose.....	25
6.6 CI/CD cevovod (GitHub Actions).....	25
6.7 Dodajanje novih funkcionalnosti - vzorec razvoja.....	25

7. Odprte pomanjkljivosti, nedoslednosti in napake ob predaji rešitve.....	27
7.1 Znane pomanjkljivosti.....	27
7.2 Tehnični dolg.....	27
8. Nadaljnji razvoj.....	28
8.1 Kratkoročne izboljšave (0–3 mesece).....	28
8.2 Srednjeročne izboljšave (3–9 mesecev).....	28
8.3 Dolgoročne izboljšave (9+ mesecev).....	28
8.4 Predlog za prevzem rešitve.....	28

1. Povzetek in opis rešitve

1.1 Namen in kontekst projekta

SGC Navigator je spletna aplikacija, katere primaren namen je zagotoviti interaktivni grafični navigator za Splošni geslovnik COBISS (SGC) - kontrolirani tezaver slovenskega knjižničnega informacijskega sistema COBISS.SI. Sistem vsebuje okrog 75.000 SKOS (Simple Knowledge Organization System) izrazov, organiziranih v kompleksno hierarhično in asociativno mrežo. Pred nastankom te rešitve ni obstajal noben specializiran vizualni vmesnik, ki bi knjižničarjem in uporabnikom sistema COBISS Plus omogočal pregledno in interaktivno navigacijo po tej obsežni strukturi pojmov.

SGC Navigator rešuje konkretno praktično težavo: geslovnik COBISS je v svoji izvorni obliki dostopen zgolj prek tabelarnih ali tekstovnih vmesnikov, kar onemogoča intuitivno razumevanje semantičnih razmerij med pojmi. Naša rešitev transformira ta statičen nabor podatkov v dinamičen, vizualno bogat interaktivni graf, v katerem lahko uporabnik v realnem času razišče hierarhična razmerja (nadrejeni in podrejeni pojmi) ter asociativne povezave (sorodni pojmi) med katerikoli izmed 75.000 konceptov.

SGC Navigator

SGC – Splošni geslovnik COBISS.SI

Iščite in raziskujte med približno 75.000 gesli SGC prek interaktivnih hierarhičnih in asociativnih razmerij.

Primeri: Medeninski predmetiMedverski odnosiMedicinske osebeMednarodna skupnost

75.000+

Nadzorovani pojmi

V realnem času

Priestkovanje vozilic slovica

Takojsnja

Dvojezična slovica

Interaktivni grafični prikazi

Doživite razmerja naravno. Naši izrisovalniki grafov s silami gladko animira povezave med sorodnimi pojmi v realnem času.

Hierarhična taksonomija

Intuitivno brskajte po vertikalnih drevesih taksonomske hierarhije.

Izvorna dvojezična preslikava

Tekoče in brezhibno prevajanje med slovenskimi in angleškimi lokaliziranimi izrazi.

Umetnost

Art

O SGC Navigatorju

SGC Navigator ponuja napreden semantični vizualni vmesnik za obsežen slovenski tezaver. S pomočjo tehnologije grafov pretvori stotine vnosov nadzorovanega besedišča v uporabniku prijazno obliko, ki natančno prikazuje strukturo in jezikovno povezanost pojmov.

Namenjen je podpori raziskovalcem, indeksirjem in razvijalcem, ki delajo s strukturiranimi bibliografskimi podatki.

© 2026 SGC Navigator. Vse pravice pridržane.

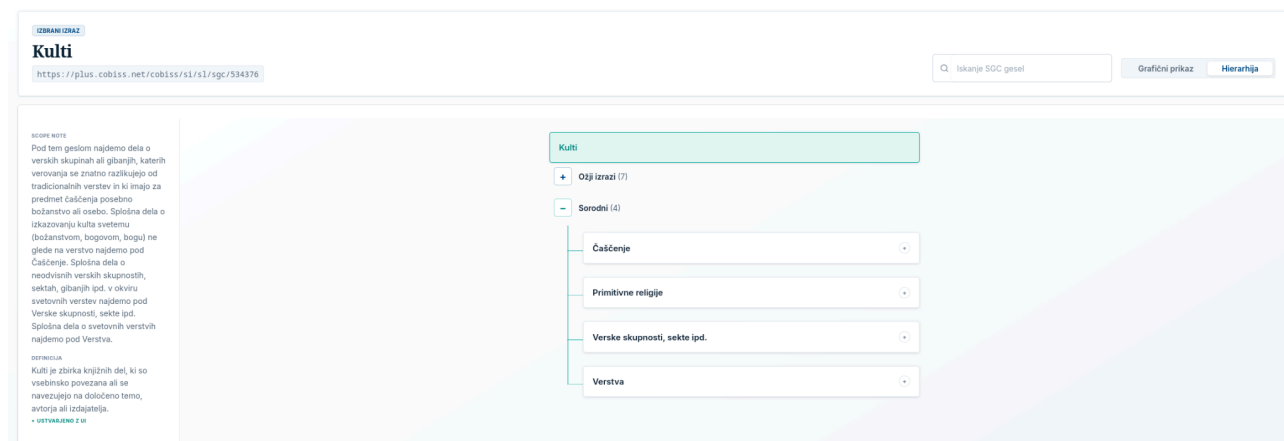
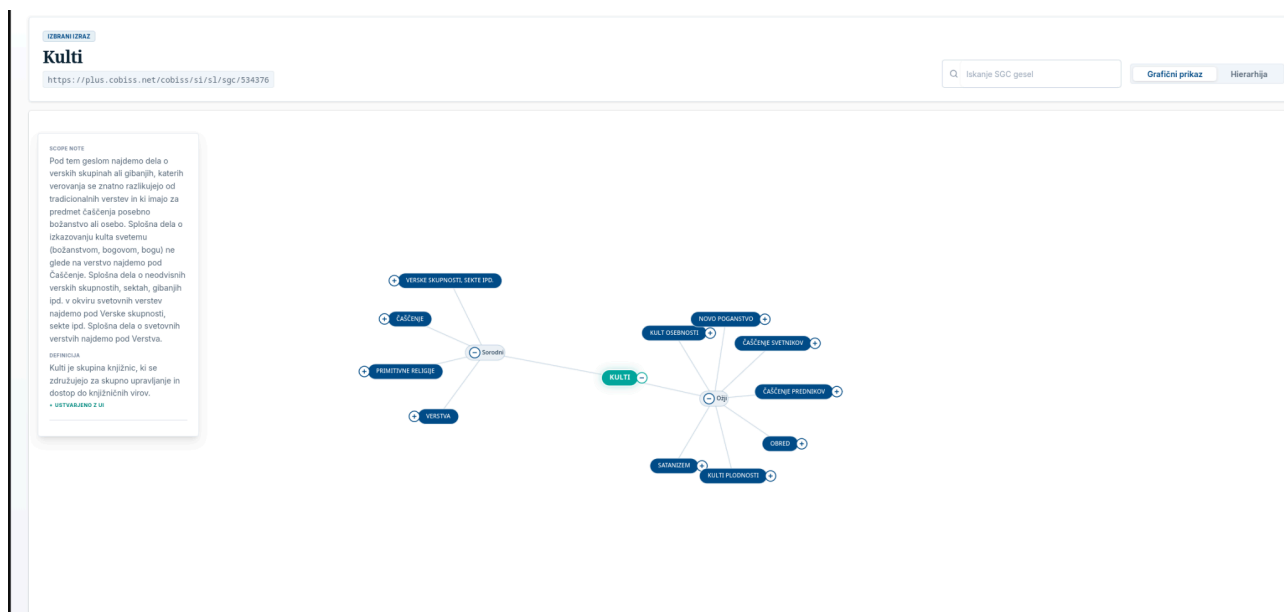
1.2 Opisana problematika in rešitev

Stran 4 od 32

Tezavri so kompleksni semantični instrumenti, ki so v knjižničnih sistemih ključni za predmetno katalogizacijo, iskanje po vsebini in medjezikovne premostitve. SGC vsebuje pojme v dveh jezikih (slovenščina in angleščina) z naslednjimi vrstami razmerij:

- **Nadrejeni pojmi (broader):** Pojmi, ki so splošnejši od danega koncepta - npr. "Računalniki" je nadrejeni pojem "Prenosnih računalnikov".
- **Podrejeni pojmi (narrower):** Pojmi, ki so bolj specifični od danega koncepta - specializacije nadrejenega.
- **Sorodni pojmi (related):** Pojmi, ki so asociativno povezani, a niso v hierarhičnem razmerju - npr. "Programiranje" in "Algoritmi".

SGC Navigator razrešuje to problematiko z dvodelnim grafičnim vmesnikom: (1) organskim gravitacijskim grafom, ki prikaže vse neposredne sosedje koncepta kot fizičen simulacijski model, in (2) hierarhično drevesno strukturo, ki ponuja pregledno klasifikacijsko navigacijo. Oba prikaza sta v živo usklajena z istimi podatki, pridobljeni prek GraphQL API-ja iz zalednega sistema, ki temelji na podatkovni bazi grafov Neo4j.



1.3 Ciljni uporabniki

Aplikacija je primarno namenjena naslednjim skupinam uporabnikov:

- **Knjižničarji in katalogizatorji:** Strokovnjaki, ki pri svojem delu vsakodnevno vzdržujejo in aplicirajo gesla iz SGC pri katalogizaciji knjižničnega gradiva. Navigatorjev vizualni vmesnik jim omogoča hitrejše razumevanje razmerij med gesli in bolj natančno izbiro pravega pojma.
- **Uporabniki COBISS Plus:** Splošni uporabniki slovenskega knjižničnega sistema, ki iščejo bibliografsko gradivo in bi radi razumeli, katere pojme sistema bi morali vključiti v svoja iskanja.
- **Razvijalci in raziskovalci:** Tehničnim profilom, ki delajo s strukturiranimi bibliografskimi podatki, SKOS ontologijami ali grafikoni znanja.

1.4 Primerjava s sorodnimi rešitvami

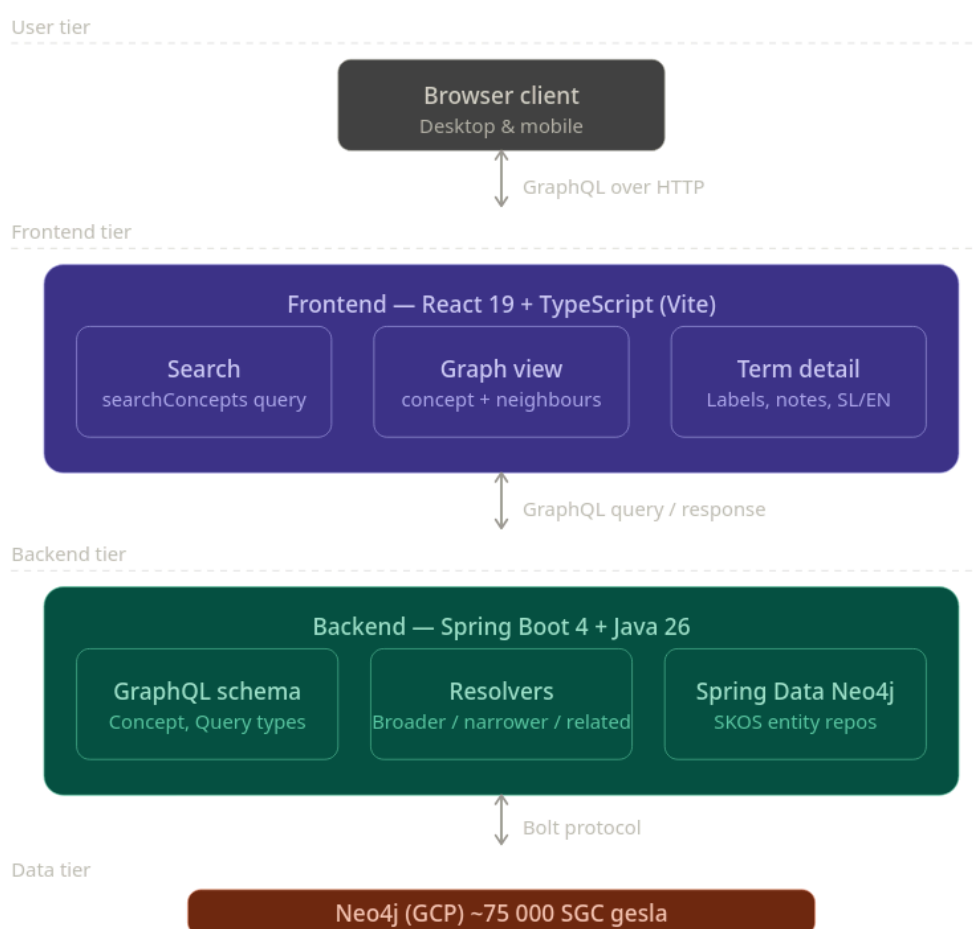
Podobna rešitev na mednarodnem nivoju je GND Explorer (<https://explore.gnd.network/>), ki vizualizira nemški nacionalni tezaver GND (Gemeinsame Normdatei). SGC Navigator ponuja primerljivo funkcionalnost, prilagojeno za slovensko okolje in COBISS infrastrukturo, z dodano vrednostjo dveh vzporednih načinov vizualizacije (gravitacijski graf + hierarhija) in podporo za slovensko-angleško dvojezičnost.

2. Arhitektura sistema

2.1 Pregled celotne arhitekture

SGC Navigator temelji na klasični trikolejni (three-tier) spletni arhitekturi, ki jo sestavljajo:

- **Predstavitveni nivo (Frontend):** Spletna aplikacija React, ki jo gostujemo na GitHub Pages pod potjo /frontend/.
- **Logični nivo (Backend API):** Spring Boot strežnik, ki gostuje GraphQL in REST API ter je nameščen na Google Cloud Platform (GCP).
- **Podatkovni nivo (Baza podatkov):** Neo4j grafična baza podatkov, ki shranjuje ~75.000 SKOS konceptov in njihove medsebojne relacije.



Komunikacija med nivojema frontend in backend poteka izključno prek dveh protokolov:

- **GraphQL (POST /graphql):** Za vse operacije pridobivanja konceptualnih podatkov - iskanje, pridobivanje posameznega koncepta, soseščine in robov grafa.
- **REST HTTP (GET/POST /api/auth/*):** Izključno za ALTCHA CAPTCHA mehanizem zaščite pred prekomerno obremenitvijo.

2.2 Odjemalska stran (Frontend)

Frontend je zgrajen kot enostranska aplikacija (Single Page Application - SPA) z ogrodjem React.

Vse poti aplikacije se začnejo z `/frontend/` in so upravljane na strani odjemalca z React Router. Dve ključni strani sta:

- **SearchPage (`/frontend/`):** Vstopna stran z iskalno vrstico, prikazom popularnih izrazov (Trending), statistikami in opisom funkcionalnosti. Izvede GraphQL poizvedbo `searchConcepts` ob vsaki spremembi iskalnega niza.
- **GraphPage (`/frontend/graph/:uri`):** Stran posameznega koncepta, ki vsebuje dva vzporedna prikaza (Graph View in Hierarchy) ter plošče z definicijami in sorodnimi pojmi. Prikaz se parametrizira z URI-jem koncepta, ki je URL-encoded v poti.

Upravljanje stanja je lokalno, implementirano prek React hookov (`useState`, `useMemo`, `useCallback`, `useEffect`). Apollo Client vzdržuje predpomnilnik (`InMemoryCache`) za GraphQL odgovore, kar prepreči podvajanje omrežnih zahtev.

2.2.1 Konfiguracija Apollo Client in proxy logika

Apollo Client se konfigurira v datoteki `apollo-client.ts`. V razvojnem načinu (`npm run dev`) vse zahteve prehajajo prek Vite proxy strežnika, ki jih posreduje na `VITE_API_URL`. V produkciji Apollo Client neposredno kliče konfigurirani `VITE_API_URL`. Ta pristop odpravlja CORS težave v razvoju, medtem ko produkcijsko okolje zahteva pravilno konfiguracijo CORS na zalednem strežniku.

2.2.2 Omejevanje dostopa (Rate Limiting) in CAPTCHA

Odjemalski mehanizem omejevanja dostopa je implementiran v `RateLimitContext.tsx` kot React Context Provider. Shranjuje časovne žige zadnjih zahtev v ref (`requestTimestampsRef`) in jih primerja z oknom 60 sekund. Ko število zahtev preseže prag 10, se prikaže CAPTCHA modal (`CaptchaModal.tsx`) z ALTCHA widgetom. Po uspešni rešitvi se `isVerified` nastavi na `true` v `sessionStorage` in vse nadaljnje zahteve so neomejene za trajanje seje.

2.3 Zaledna stran (Backend)

Backend temelji na Spring Boot z moduli Spring Data Neo4j in Spring for GraphQL. Strukturiran je po standardnem vzorcu slojevite arhitekture (layered architecture):

- **Nivo krmilnikov (Controllers):** `ConceptGraphQLController.java` obravnava vse GraphQL poizvedbe in field-level resolverje. `GatewayController.java` obravnava REST končne točke za CAPTCHA.
- **Nivo storitev (Services):** `ConceptService.java` vsebuje vso poslovno logiko in delegira na repozitorij ter `Neo4jClient`. `AltchaService.java` implementira generiranje in preverjanje CAPTCHA izzivov.
- **Nivo repozitorija (Repository):** `ConceptRepository.java` razširja `Neo4jRepository` in definira Cypher poizvedbe z anotacijo `@Query`.
- **Nivo modelov (Models):** `SkosConcept.java`, `SkosConceptScheme.java`, `ConceptProjection.java` (vmesnik), `ConceptEdge.java` (Java record), `Resource.java`.

2.4 Podatkovna baza Neo4j in SKOS podatkovni model

Neo4j je grafična baza podatkov, ki shranjuje entitete kot vozlišča (nodes) in razmerja med njimi kot usmerjene povezave (relationships). To jo naredi idealno za SKOS tezavre, kjer so razmerja med pojmi prav tako pomembna kot pojmi sami.

SKOS koncepti so shranjeni kot vozlišča z oznako `skos__Concept` (skupaj z oznako `Resource`). Lastnosti vozlišč sledijo SKOS imenski shemi z dvojnim podčrtajem (double-underscore) kot ločilnikom imenskega prostora:

- **uri**: Enolični identifikator koncepta - npr. "http://thesaurus.cobiss.si/concept/12345".
- **skos__prefLabel**: Seznam nizov z jezikovnimi oznakami - npr. ["računalnik@sl", "computer@en"].
- **skos__altLabel**: Seznam alternativnih oznak (sinonimov) z jezikovnimi oznakami.
- **skos__definition**: Seznam nizov z definicijo koncepta z jezikovnimi oznakami.
- **skos__scopeNote**: Opomba o obsegu koncepta (kontekstualna razlaga).

Razmerja med vozlišči so shranjena kot direktne SKOS relacije:

- **skos__narrower (OUTGOING)**: Iz nadrejenega koncepta → k podrejenemu konceptu.
- **skos__broader (ODHODNO)**: Iz podrejenega koncepta → k nadrejenemu konceptu.
- **skos__related (OUTGOING)**: Asociativna, nedirektivna semantična povezava.
- **skos__inScheme (OUTGOING)**: Uvrstitev koncepta v shemo `ConceptScheme`.

Ključni vzorec poizvedbe za ekstrakcijo oznak po jeziku: `[lbl IN n.skos__prefLabel WHERE lbl ENDS WITH "@sl"]` - ta Cypher izraz filtrira seznam oznak po jezikovni priponi in vrne prvo ujemajočo se vrednost.

2.5 GraphQL API shema

GraphQL shema definira točno pogodbo med frontendov in backendom. Celotna shema je definirana v datoteki `schema.graphqls` in vsebuje naslednje tipe:

```
type Query {
  concept(uri: String!): Concept
  schemes: [ConceptScheme]
  searchConcepts(text: String!, limit: Int = 20, lang: String = "sl"): [Concept]
  conceptNeighborhood(uri: String!): [Concept!]!
  conceptNeighborhoodEdges(uri: String!): [ConceptEdge!]!
}

type Concept {
  uri: String!
  prefLabel: String          # Slovensko z angleškim nadomestkom (rešeno strežniško)
  prefLabelSl: String         # Čista slovenska oznaka
  prefLabelEn: String         # Čista angleška oznaka
  definition: String
  scopeNote: String
  broader: [Concept]          # Lenobno razrešeno prek @SchemaMapping
  narrower: [Concept]         # Lenobno razrešeno prek @SchemaMapping
  related: [Concept]          # Lenobno razrešeno prek @SchemaMapping
}

type ConceptEdge {
  sourceUri: String!
  targetUri: String!
  relationType: String! # "broader" | "narrower" | "related"
}

type ConceptScheme {
  uri: String!
  title: String
  description: String
  topConcepts: [Concept]
```

Polja `broader`, `narrower` in `related` so v GraphQL shemi definirana kot lenobno razrešena (`@SchemaMapping`), kar pomeni, da se zanje izvede ločena Neo4j poizvedba šele takrat, ko jih odjemalec eksplicitno zahteva. Za masovno pridobivanje celotnega grafa se raje verwendet poizvedbi `conceptNeighborhood` in `conceptNeighborhoodEdges`, ki izvedeta optimizirano Cypher poizvedbo v enem koraku.

2.6 Varnostna arhitektura - ALTCHA CAPTCHA

Za zaščito sistema pred avtomatiziranimi poizvedbami (botov in DDOS napadov) je implementiran dvodelni mehanizem:

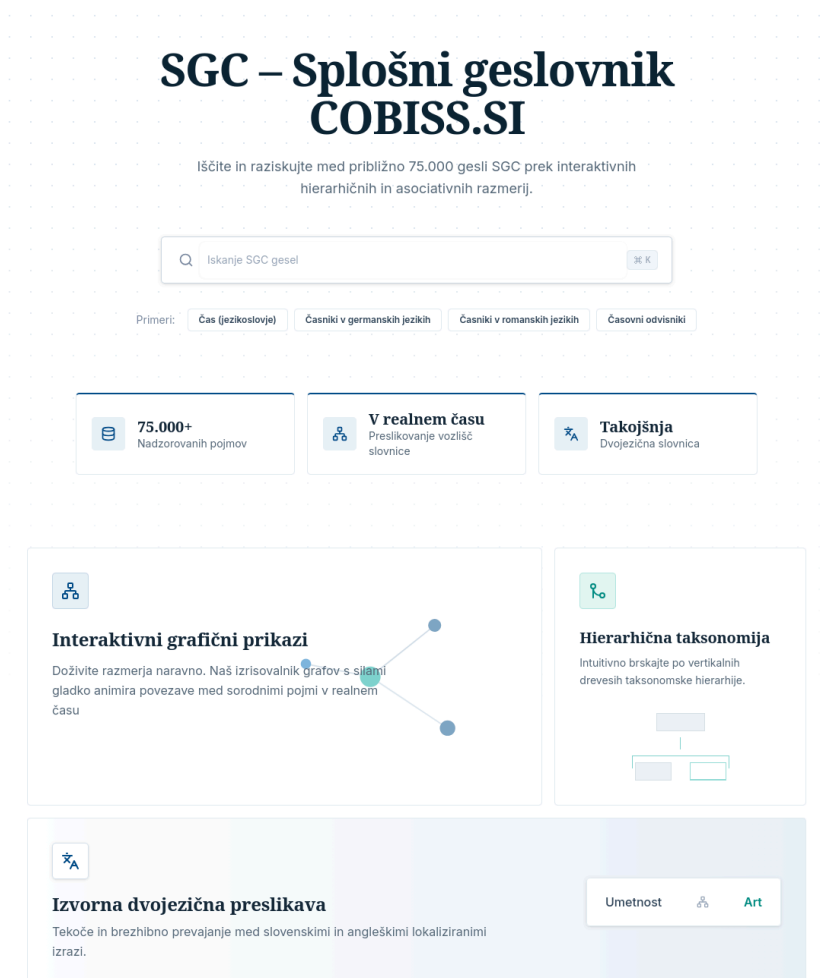
- **Odjemalska zaščita (`RateLimitContext.tsx`):** Na strani frontenda se shranjuje seznam časovnih žigov zadnjih 10 zahtev. Kadar v 60-sekundnem oknu pride do 10. zahteve, se samodejno prikaže ALTCHA CAPTCHA modal.
- **Strežniška zaščita (`AltchaService.java`):** Backend generira SHA-256 uganko z HMAC podpisom in rokom veljavnosti 5 minut (GET `/api/auth/captcha-challenge`). Ko odjemalec ugodbo reši, pošlje Base64-kodirani odgovor na POST `/api/auth/verify-gateway`, kjer strežnik preveri: (1) rok veljavnosti soli, (2) HMAC podpis, (3) pravilnost rešene uganke.

3. Struktura uporabniških vmesnikov

3.1 Vstopna stran - SearchPage

SearchPage je osnovna vstopna točka aplikacije, ki se prikaže na poti /frontend/. Stran je zasnovana kot "iskalni portal" z jasno hierarhijo vizualnih elementov:

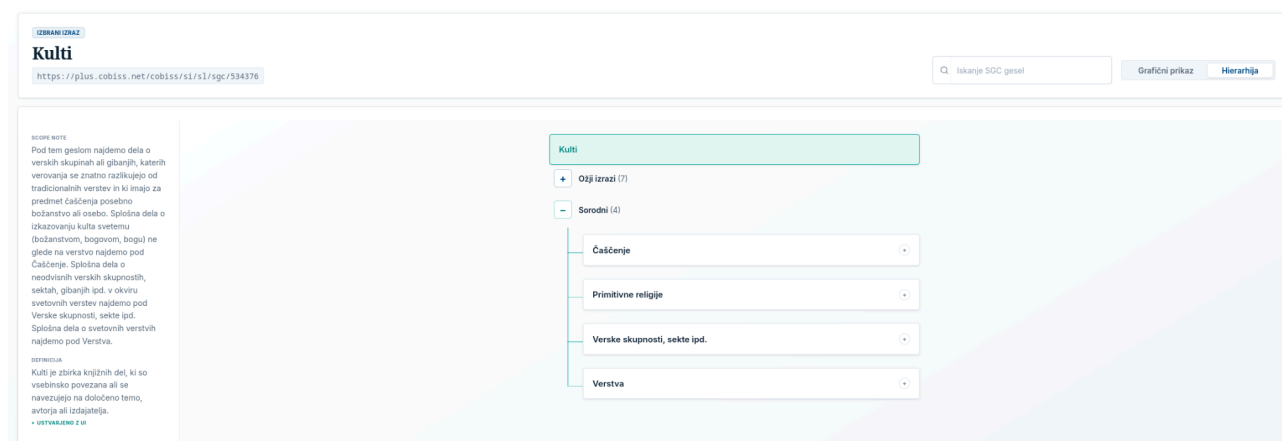
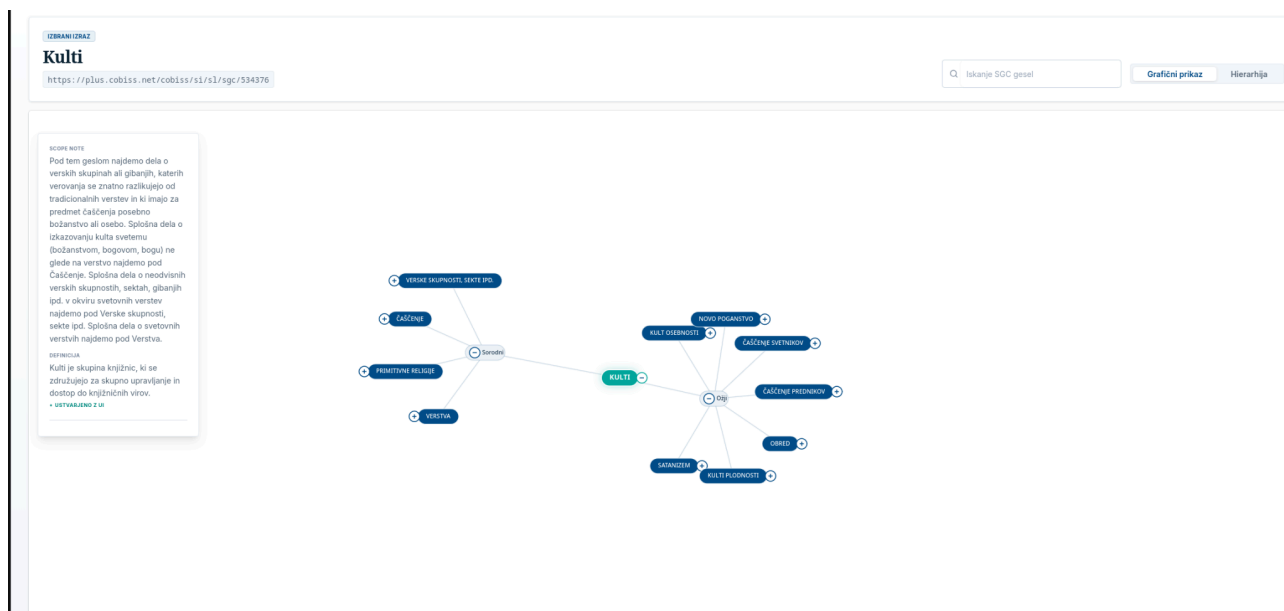
- **Navigacijska vrstica (Header):** Z logotipom SGC Navigator, preklopnikom tema (svetla/temna) in menjalnikom jezika (SL/EN). Vsebuje tudi indikator stanja CAPTCHA preverjanja.
- **Hero section:** Velik naslov "Explore the Semantic Web", podnaslov in osrednja iskalna vrstica z mrežnim ozadjem.
- **Sekcija "Primeri":** Dinamično prikazuje 4 naključno izbrane pojme iz baze, ki se menjajo ob vsakem nalaganju strani. Pojmi so pridobljeni prek GraphQL poizvedbe searchConcepts z naključno začetno besedo (seed).
- **Statistična pasica:** Trije bloki s ključnimi podatki: "75.000+ Nadzorovanih pojmov", "V realnem času preslikovanje vozlišč slovnice", "Takojšnja dvojezična slovnica".
- **Mreža funkcionalnosti (Feature Grid):** Trije kartonski bloki z animiranimi SVG maketi, ki opisujejo funkcionalnosti: Interaktivni grafični prikazi, Hierarhična taksonomija in Izvorna dvojezična preslikava.
- **O aplikaciji (About block):** Informativen odstavek o SGC Navigatorju z opisom namena in ciljnih skupin.



3.2 Stran koncepta - GraphPage

GraphPage se prikaže ob navigaciji na pot /frontend/graph/:uri in je osrednji funkcionalni del aplikacije. Stran vsebuje:

- **Orodjarna (Toolbar):** Prikazuje ime izbranega koncepta, njegov URI v monospace obliki, iskalno vrstico za navigacijo na drug koncept in dve stikali za preklapljanje med zavihkoma Graf in Hierarhija.
- **Zavihek Graf (Graph View):** Prikazuje interaktivni gravitacijski graf z react-force-graph-2d, narisanim na Canvas elementu. Vsebuje definicijsko ploščo na levi (overlay) z opredelitvijo in opombo o obsegu.
- **Zavihek Hierarhija (Hierarchy):** Prikazuje hierarhično drevo z nadrejenimi, podrejenimi in sorodnimi pojmi v collapsible seznamih. Na desktopskem pogledu ima ploščo z definicijo na levi strani.



3.3 Navigacija in URL parametri

Aplikacija implementira URL-based state management. Zavihek (graf ali hierarhija) je shranjen kot URL parameter ?tab=graph ali ?tab=hierarchy. URI koncepta je URL-kodiran v poti. To pomeni, da so vsi pogledi aplikacije globoki linki (deep links) - posamezno stanje je mogoče direktno deliti ali zaznamovati.

Primer URL-a za hierarhični prikaz koncepta z URI-jem <http://thesaurus.cobiss.si/concept/123>:

```
/frontend/graph/http%3A%2F%2Fthesaurus.cobiss.si%2Fconcept%2F123?tab=hierarchy
```

4. Zunanje odvisnosti aplikacije

4.1 Frontend - odvisnosti

Naslednja tabela navaja vse ključne knjižnice in ogrodja, ki jih verwendet frontend aplikacija, skupaj z razlago njihovega namena:

Knjižnica / Ogrodje	Namen in razlaga	Vloga v projektu
React 19 + React DOM	Temeljno ogrodje za gradnjo uporabniškega vmesnika z deklarativnimi komponentami in hookami (useState, useEffect, useMemo, useCallback, useRef, useLayoutEffect).	Runtime okolje SPA
TypeScript 6.x	Stroga tipizacija nad JavaScriptom. Vse komponente, interfeisi in funkcije so polno tipizirani - npr. interfeisi ConceptNode, GraphNode, GraphLink, CanvasPalette.	Varnost tipov, boljša vzdrževanost
Vite 8.x	Hitrostni gradilni sistem in razvojni strežnik z HMR (Hot Module Replacement) in Oxc transpilacijo. Konfigurira proxy za /graphql in /api v razvoju.	Razvojna produktivnost
React Router 7.x	Upravljanje URL poti v SPA. Definira route /frontend/ in /frontend/graph/:uri, URL parametre koncepta in navigacijo med pojmi.	Klientski routing
Apollo Client 4.x	GraphQL odjemalec z InMemoryCache, authLink za bearer token in httpLink. Zagotavlja useQuery, useLazyQuery in useApolloClient hooke.	GraphQL komunikacija
Tailwind CSS 4.x	Utility-first CSS ogrodje za stiliranje komponent z razredi kot je bg-[var(--brand-navy)], text-foreground ipd. Integriran prek @tailwindcss/vite vtičnika.	Stiliranje UI
shadcn/ui	Zbirka dostopnih, stiliziranih UI komponent (Card, Badge, Tabs, Skeleton, Separator), ki temeljijo na Radix UI primitives.	Prebuilt UI komponente
lucide-react	Knjižnica SVG ikon (Network, GitMerge, Languages, Database) za prikaz v UI komponentah.	Ikone
react-force-graph-2d	Wrapper nad D3 force simulation za Canvas-based prikaz grafov z vozlišči in robovi. Ponuja ForceGraph2D komponento z nodeCanvasObject, onNodeClick, onNodeHover.	Gravitacijski graf
d3-force	D3.js modul za simulacijo fizikalnih sil (forceCharge, forceLink, forceCollide) pri urejanju vozlišč grafa.	Fizikalna simulacija grafa
@xyflow/react (ReactFlow)	Namenjeno za hierarhični prikaz grafov (zaenkrat v planu, nakazano v README), alternativa k react-force-graph za drevesni prikaz.	Hierarhični prikaz
framer-motion v12.x	Animacijska knjižnica za prehode med stranmi (AnimatePresence v App.tsx) in animirane komponente.	Animacije prehodov
react-helmet-async	Dinamično upravljanje <head> sekcije HTML dokumenta - title, meta, og: oznake in canonical URL za SEO optimizacijo.	SEO upravljanje

Knjižnica / Ogradje	Namen in razlaga	Vloga v projektu
<code>i18next + react-i18next</code>	Internacionalizacijska knjižnica z datotekami prevodov v <code>public/locales/{sl,en}/translation.json</code> . Podpira spremembo jezika v živo.	Večjezičnost (SL/EN)
<code>altcha</code>	ALTCHA CAPTCHA knjižnica - custom HTML element <code>altcha-widget</code> za prikaz CAPTCHA izziva na odjemalcu.	Zaščita pred boti
<code>Vitest 4.x</code>	Enotno testno okolje (unit testing), kompatibilno z Vite. Zahteva jsdom za simulacijo brskalnikovega DOM.	Testiranje
<code>@testing-library/react</code>	Utiliteti za renderiranje React komponent v testih z DOM-first pristopom.	Testiranje komponent

4.2 Backend - odvisnosti

Zaledni sistem temelji na Java 21 LTS in Spring Boot ekosistemu. Naslednja tabela opisuje ključne odvisnosti:

Tehnologija	Namen in razlaga	Vloga v projektu
<code>Spring Boot 4.0.6</code>	Temeljno Java ogrodje za gradnjo podjetniških spletnih aplikacij z avtomatsko konfiguracijo, vbrizgavanjem odvisnosti (DI) in vgrajenim Tomcat strežnikom.	Aplikacijski okvir
<code>Spring Data Neo4j</code>	Integracija med Spring in Neo4j grafično bazo. Zagotavlja <code>@Node</code> , <code>@Relationship</code> anotacije, <code>Neo4jRepository</code> in <code>Neo4jClient</code> za Cypher poizvedbe.	OGM (Object-Graph Mapping)
<code>Spring for GraphQL</code>	Spring modul za gradnjo GraphQL API-jev. Zagotavlja <code>@QueryMapping</code> , <code>@SchemaMapping</code> anotacije in avtomatsko mapiranje med GraphQL shemo in Java metodami.	GraphQL strežnik
<code>Gradle 8</code>	Sistem za gradnjo Java projektov z Java 21 toolchain konfiguracijo. Upravlja odvisnosti, testiranje in pakiranje v JAR datoteko.	Sistem gradnje
<code>Java 21 (LTS)</code>	Najnovejša dolgoročno podprta verzija Jave z modernimi funkcionalnostmi: records, sealed classes, pattern matching in virtual threads.	Programski jezik
<code>Lombok</code>	Anotacijski procesor za samodejno generiranje konstruktorjev, getterjev, setterjev, <code>toString</code> in <code>equals</code> metod (<code>@Data</code> , <code>@Getter</code> , <code>@NoArgsConstructor</code>).	Zmanjšanje kode
<code>dotenv-java 3.0.0</code>	Knjižnica za nalaganje spremenljivk okolja iz <code>.env</code> datoteke pred zagonom Spring konteksta. Zagotavlja konsistentno delovanje v lokalnem in Docker okolju.	Upravljanje konfiguracije
<code>Neo4j (Graph DB)</code>	Grafična baza podatkov shranjevanja SKOS konceptov in njihovih relacij. Izpostavlja Bolt protokol na portu 7687 in browser UI na 7474.	Shramba podatkov

Tehnologija	Namen in razlaga	Vloga v projektu
ALTCHA (server)	Serverska implementacija ALTCHA protokola - generiranje SHA-256 izzivov z HMAC podpisom in rokom veljavnosti.	CAPTCHA mehanizem
Springdoc OpenAPI	Samodejno generiranje Swagger UI dokumentacije za REST končne točke na /swagger-ui in /api-docs.	API dokumentacija
Jackson (ObjectMapper)	JSON serializacija/deserializacija v Java objektih, vključno s parsiranjem Base64-kodiranega ALTCHA odgovora v AltchaService.	JSON procesiranje
JUnit 5 (Jupiter)	Testno ogrodje za pisanje in izvajanje enotinih in integracijskim testov. @SpringBootTest za test nalaganja celotnega konteksta.	Testiranje
Docker (multi-stage)	Večstopenjska Docker gradnja: faza gradnje z gradle:8-jdk21, faza zagona z eclipse-temurin:21-jre-jammy za minimalno produkcijsko sliko.	Kontejnerizacija

5. Izseki pomembne kode

5.1 GraphQL shema - schema.graphqls

GraphQL shema je pogodba med frontendov in backendom. Definira vse razpoložljive poizvedbe, tipe podatkov in argumente. Shema je napisana v GraphQL Schema Definition Language (SDL) z dokumentacijskimi komentarji v slovenščini (med trojnimi narekovaji """). Naslednji izsek prikazuje ključne dele sheme:

```
"""
Iskanje po besedilu čez oznake konceptov.
Rezultati so razvrščeni po kakovosti ujemanja (točno > začetek > vsebuje).
"""
searchConcepts(
  text: String!
  limit: Int = 20
  """
  Jezikovni filter: "sl" (privzeto), "en", "both"
  """
  lang: String = "sl"
): [Concept]

"""
Vrne vsa vozlišča v neposredni soseščini koncepta -
vključno z nadrejenimi, podrejenimi in sorodnimi ter njihovimi sorodnimi.
"""
conceptNeighborhood(uri: String!): [Concept!]!

conceptNeighborhoodEdges(uri: String!): [ConceptEdge!]!
```

Shema uvaja ključno razlikovanje med dvema načinoma pridobivanja relacij: posamičnimi field-level resolverji (broader, narrower, related na tipu Concept) in skupinsko optimiziranim parom poizvedb (conceptNeighborhood + conceptNeighborhoodEdges). Slednji par je namenjen za GraphPage, ker z eno Cypher poizvedbo pridobi celoten graf v enem obhodu baze, kar prepreči problem N+1 poizvedb.

5.2 Podatkovni model - SkosConcept.java

Razred SkosConcept je JPA-podobna entiteta za Neo4j, ki se preslika na vozlišča z oznako skos__Concept. Klic `@Node({"skos__Concept", "Resource"})` pove Spring Data Neo4j, da naj ta Java razred preslikuje na vozlišča, ki imajo obe oznaki v Neo4j.

```
@Node({"skos__Concept", "Resource"})
@Getter @Setter @NoArgsConstructor
public class SkosConcept extends Resource {

    private String prefLabelSl;    // Razrešena slovenska oznaka
    private String prefLabelEn;    // Razrešena angleška oznaka
    private String definition;
    private String scopeNote;

    @Property("skos__prefLabel")
    private List<String> rawPrefLabels; // Surovi podatki: ["računalnik@sl", "computer@en"]

    @Property("skos__altLabel")
    private List<String> altLabel;

    @Relationship(type = "skos__related", direction = OUTGOING)
    private List<SkosConcept> related;

    @Relationship(type = "skos__broader", direction = OUTGOING)
    private List<SkosConcept> broader;

    @Relationship(type = "skos__narrower", direction = INCOMING)
    private List<SkosConcept> narrower;

    // Rezervna strategija: sl ima prednost pred en
    public String getPrefLabel() {
        return this.prefLabelSl != null ? this.prefLabelSl : this.prefLabelEn;
    }
}
```

Ključna opomba: relacija skos__narrower ima smer INCOMING, ker je v Neo4j usmerjena iz nadrejenega koncepta (broader) proti podrejenemu (narrower). Ko torej iščemo "podrejene" pojme danega koncepta, dejansko iščemo dohodne puščice tipa skos__narrower.

5.3 Projekcijski vmesnik - ConceptProjection.java

Spring Data Neo4j podpira projekcije vmesnikov (interface projections) kot zmogljiv mehanizem za vračanje samo izbranih polj iz Cypher poizvedbe brez polne materializacije entitetnih objektov. ConceptProjection definira pogodbene getter metode, ki se samodejno preslikajo iz aliasov Cypher poizvedbe:

```
public interface ConceptProjection {
    String getUri();
    String getPrefLabelSl();
    String getPrefLabelEn();
    String getScopeNote();
    String getDefinition();
    List<String> getAltLabel();

    // Rekurzivne projekcije - varno gnezdenje
    List<ConceptProjection> getBroader();
    List<ConceptProjection> getNarrower();
    List<ConceptProjection> getRelated();

    List<String> getRawPrefLabels();

    // Privzeta metoda Java 8+ - ne zahteva implementacije
    default String getPrefLabel() {
        return getPrefLabelSl() != null ? getPrefLabelSl() : getPrefLabelEn();
    }
}
```

Zakaj projekcija namesto entitete? Ker Spring Data Neo4j pri polni materializaciji entitetnega objekta (SkosConcept) avtomatično nalaga vse relacije, kar bi pri 75.000 konceptih pomenilo eksplozijo poizvedb. Z vmesniškimi projekcijami dobimo samo tisto, kar je definirano v Cypher stavku RETURN, kar je bistveno bolj učinkovito.

5.4 DTO za robove grafa - ConceptEdge.java

ConceptEdge je minimalen Java record (nova funkcionalnost Java 16+), ki prenaša podatke o posamezni usmerjeni povezavi v grafu. Java records so nemutabilni in samodejno generirajo konstruktor, getter metode, equals, hashCode in toString.

```
public record ConceptEdge(String sourceUri, String targetUri, String relationType) {}
```

Kljub svoji minimalnosti je ta razred ključen - vsak objekt ConceptEdge predstavlja eno usmerjeno relacijo v grafu (npr. sourceUri="http://.../123", targetUri="http://.../456", relationType="narrower"). Frontend s tem podatki sestavi vizualizacijo grafa.

5.5 Cypher poizvedbe v ConceptRepository.java

ConceptRepository vsebuje vse Cypher poizvedbe, anotirane z @Query. Ključni vzorec, ki se pojavlja v vseh poizvedbah, je ekstrakcija oznak po jeziku:

```
// Ekstrakcija slovenske oznake iz seznama z jezikovnimi priponi
[lbl IN n.skos__prefLabel WHERE lbl ENDS WITH "@sl"][0] AS raw_sl

// Ekstrakcija angleške oznake
[lbl IN n.skos__prefLabel WHERE lbl ENDS WITH "@en"][0] AS raw_en

// Ekstrakcija definicije s slovenskim nadomestkom
coalesce([d IN n.skos__definition WHERE d ENDS WITH "@sl"][0],
         n.skos__definition[0]) AS definition
```

Poizvedba findNeighborhood(uri) je najpomembnejša in najzahtevnejša - vrne celotno 1-hop sosesčino koncepta v eni Cypher transakciji:

```
@Query("MATCH (n:skos__Concept {uri: $uri}) WITH n " +
"OPTIONAL MATCH (n)<-[:skos__narrower]-(broader) " +
"OPTIONAL MATCH (n)-[:skos__narrower]->(narrower) " +
"OPTIONAL MATCH (n)-[:skos__related]->(related) " +
"WITH n, collect(DISTINCT broader) AS broaderList, " +
"collect(DISTINCT narrower) AS narrowerList, " +
"collect(DISTINCT related) AS relatedList " +
"OPTIONAL MATCH (broaderNode)-[:skos__related]->(broaderRelated) " +
"WHERE broaderNode IN broaderList " +
// ... nato unwind in return vseh vozlišč",
)
List<ConceptProjection> findNeighborhood(String uri);
```

Ista logika je v ConceptService.java podvojena za getNeighborhoodEdges() z Neo4jClient za direktne Cypher poizvedbe, ker Spring Data @Query ne podpira UNION stavkov, ki so tu potrebni za vračanje robov vseh vrst relacij v eni poizvedbi.

5.6 ConceptService - logika iskanja po jeziku

ConceptService.searchConcepts() implementira trirazredno logiko iskanja glede na izbrani jezik. To je primer Java 21 switch expression za elegantno izbiro med repozitorijsko metodo:

```
public List<ConceptProjection> searchConcepts(String text, int limit, String lang) {
    return switch (lang.toLowerCase()) {
        case "en" -> conceptRepository.searchByTextEn(text, limit);
        case "both" -> conceptRepository.searchByText(text, limit);
        default -> conceptRepository.searchByTextSl(text, limit);
    };
}
```

Poizvedba searchByTextSl razvršča rezultate po kakovosti ujemanja s pomočjo CASE izreka v Cypher: točno ujemanje (score=0) ima prednost pred začetnim ujemanjem (score=1), ki ima prednost pred vsebinskim ujemanjem (score=2). To zagotavlja, da se pri iskanju "računalnik" najprej pokaže točen zadetek, nato pojmi, ki se začnejo z "računalnik", in nazadnje tisti, ki besedo vsebujejo nekje v sredini.

```
"CASE " +
"WHEN toLower(raw_sl) = toLower($text) THEN 0 " +
"WHEN toLower(raw_sl) STARTS WITH toLower($text) THEN 1 " +
"ELSE 2 " +
"END AS score " +
"ORDER BY score ASC LIMIT $limit"
```

5.7 ConceptGraphQLController - field-level resolverji

ConceptGraphQLController je osrednji GraphQL krmilnik. Metode z @QueryMapping ustrezajo koreni poizvedbi v GraphQL shemi. Metode z @SchemaMapping(typeName = "Concept", field = "...") so lenobni (lazy) razreševalci za posamezna polja tipa Concept. Ključna je metoda prefLabel, ki implementira slovensko-angleško rezervno logiko (fallback):

```
@SchemaMapping(typeName = "Concept", field = "prefLabel")
public String prefLabel(ConceptProjection concept) {
    // 1. Prednost ima slovenska oznaka
    if (concept.getPrefLabelSl() != null) return concept.getPrefLabelSl();
    // 2. Rezervna vrednost: angleška oznaka
    if (concept.getPrefLabelEn() != null) return concept.getPrefLabelEn();
    // 3. Rezervna vrednost: surovi seznam z jezikovnimi priponi
    List<String> rawLabels = concept.getRawPrefLabels();
    if (rawLabels != null && !rawLabels.isEmpty()) {
        for (String label : rawLabels) {
            if (label.endsWith("@sl")) return label;
        }
        for (String label : rawLabels) {
            if (label.endsWith("@en")) return label;
        }
        return rawLabels.get(0);
    }
    return null;
}
```

Ta večstopenjski rezervni mehanizem zagotavlja, da se vedno prikaže neka oznaka, četudi katera od prevedenih vrednosti manjka - kar je pri 75.000 pojmih realističen scenarij, saj vsi pojmi nimajo angleškega prevoda.

5.8 GatewayController - REST API za CAPTCHA

GatewayController implementira REST API za ALTCHA CAPTCHA mehanizem z dvema končnima točkama. Vsaka metoda je bogato dokumentirana s Swagger/OpenAPI anotacijami za samodejno generiranje API dokumentacije:

```
@RestController
@RequestMapping("/api/auth")
public class GatewayController {

    @Autowired
    private AltchaService altchaService;

    @PostMapping("/verify-gateway")
    public ResponseEntity<Void> verifyGatewayElement(
        @RequestBody Map<String, String> body) {
        String payload = body.get("payload");
        if (payload == null || !altchaService.validateResponse(payload)) {
            return ResponseEntity.status(HttpStatus.FORBIDDEN).build();
        }
        return ResponseEntity.ok().build();
    }

    @GetMapping("/captcha-challenge")
    public ResponseEntity<Map<String, Object>> getChallenge() {
        return ResponseEntity.ok(altchaService.generateChallenge());
    }
}
```

Metoda verifyGatewayElement vrne HTTP 200 OK ob uspešnem preverjanju ali HTTP 403 Forbidden ob neuspešnem. Brez telesa odgovora - preprost binarni rezultat, ki je dovolj za odjemalca.

5.9 AltchaService - generiranje in preverjanje CAPTCHA

AltchaService je osrednji varnostni mehanizem. Metoda generateChallenge() ustvari uganko, ki jo mora brskalnik rešiti z brute-force SHA-256 izračunom. Ključni koraki so:

```
public Map<String, Object> generateChallenge() {
    // 1. Naključno skrivno število (uganko) med 1000 in 51000
    int number = new SecureRandom().nextInt(50000) + 1000;

    // 2. Ustvari sol z vgrajenim rokom veljavnosti (5 minut)
    String salt = Base64.getEncoder().encodeToString(SecureRandom.getSeed(12));
    long expiresAt = Instant.now().plusSeconds(300).toEpochMilli();
    String saltWithMeta = salt + "?expires=" + expiresAt;

    // 3. SHA-256 hash (sol + skrivno število) = izziv
    String challenge = bytesToHex(digest.digest((saltWithMeta + number).getBytes()));

    // 4. HMAC-SHA256 podpis soli za preprečitev nedovoljenih sprememb
    String signature = hmacSha256(saltWithMeta, secretKey);

    return Map.of("algorithm", "SHA-256", "challenge", challenge,
                  "salt", saltWithMeta, "signature", signature,
                  "maxnumber", 100000);
}
```

Metoda validateResponse() izvede tri neodvisna preverjanja: (1) ali rok veljavnosti izziva ni potekel, (2) ali je HMAC podpis soli pravilen (ni bilo poseganja v izziv), (3) ali hash(sol + predan odgovor) == izziv. Šele ko so vsa tri preverjanja uspešna, se vrne HTTP 200. Ta tristopenjska validacija je robustna in odporna na replay napade, ker sol vsebuje časovni žig veljavnosti.

5.10 WebConfig - CORS konfiguracija

CORS (Cross-Origin Resource Sharing) konfiguracija je kritična varnostna nastavitev, ki določa, kateri izvori (origins) smejo pošiljati zahteve na zaledni API. WebConfig.java jo implementira za vso poti:

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Value("${FRONTEND_URL:http://localhost:5173}")
    private String frontendUrl;

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/graphql")
            .allowedOrigins(frontendUrl)
            .allowedMethods("POST", "OPTIONS")
            .allowedHeaders("*");

        registry.addMapping("/api/**")
            .allowedOrigins(frontendUrl)
            .allowedMethods("GET", "POST", "OPTIONS")
            .allowedHeaders("*");

        registry.addMapping("/swagger-ui/**")
            .allowedOrigins(frontendUrl)
            .allowedMethods("GET", "OPTIONS")
            .allowedHeaders("*");
    }
}
```

Vrednost FRONTEND_URL se bere iz spremenljivk okolja (npr. <https://cobiss-praktikum.github.io> v produkciji). Privzeta vrednost <http://localhost:5173> zagotavlja, da razvoj deluje brez eksplicitne konfiguracije.

V produkcijskem okolju je obvezno nastaviti `FRONTEND_URL`, saj sicer CORS privzeto ne dovoli dostopa iz oddaljenih izvorov.

5.11 Vite konfiguracija - proxy za razvoj

`vite.config.ts` konfigurira razvojni proxy strežnik, ki posreduje zahteve na `/graphql` in `/api` na ciljni backend URL. Brez tega proxy-ja bi brskalnik blokiral zahteve na drug origin (port 8080) iz razvojnega strežnika (port 5173) zaradi CORS omejitev:

```
export default defineConfig(({ mode }) => {
  const env = loadEnv(mode, process.cwd(), '')
  return {
    plugins: [react(), tailwindcss()],
    server: {
      proxy: {
        '/graphql': {
          target: env.VITE_API_URL,    // npr. http://localhost:8080
          changeOrigin: true,
        },
        '/api': {
          target: env.VITE_API_URL,
          changeOrigin: true,
        }
      },
    },
    base: '/frontend/',    // GitHub Pages pot
    resolve: {
      alias: { '@': path.resolve(__dirname, './src') },
    },
  }
})
```

Nastavitev `base: "/frontend/"` je ključna za pravilno delovanje na GitHub Pages, kjer je aplikacija nameščena pod podpotjo. Brez te nastavitve bi brskalnik iskal statične datoteke na napačni poti.

5.12 RateLimitContext - odjemalska zaščita

`RateLimitContext.tsx` implementira odjemalsko omejevanje dostopa kot React Context. Hrani seznam časovnih žigov (timestamps) v `useRef`, kar prepreči nepotrebne ponovne rendere ob vsaki zahtevi. Logika je preprosta a učinkovita:

```
const RATE_LIMIT_THRESHOLD = 10; // zahtev na minuto
const TIME_WINDOW = 60000;      // ms

const checkRateLimit = useCallback(() => {
  if (isVerified) return true;    // Preverjeni uporabniki so izvzeti

  const now = Date.now();
  // Odstrani zastarele timestamps (starejše od 60s)
  requestTimestampsRef.current = requestTimestampsRef.current
    .filter(ts => now - ts < TIME_WINDOW);

  if (requestTimestampsRef.current.length < RATE_LIMIT_THRESHOLD) {
    return true;    // Pod pragom - dovoli zahtevo
  }

  setShowCaptchaModal(true);    // Sproži CAPTCHA modal
  return false;
}, [isVerified]);
```

Stanje `isVerified` se shranjuje v `sessionStorage` (`captcha_verified = true`), kar zagotavlja, da CAPTCHA po uspešnem preverjanju ni potrebna ob vsakem dostopu v isti brskalni seji, ne pa med sejami - kar je pravilen varnostni kompromis med UX in zaščito.

5.13 Frontend GraphQL poizvedbe - `GraphPage.tsx`

`GraphPage.tsx` definira dve GraphQL poizvedbi kot gql konstantni. `GET_CONCEPT` pridobi celoten koncept z vsemi relacijami (used za info panel in hierarhijo), `GET_NEIGHBORHOOD` pa pridobi optimiziran seznam vozlišč in robov za gravitacijski graf:

```
const GET_CONCEPT = gql`
  query GetConcept($uri: String!) {
    concept(uri: $uri) {
      uri prefLabel prefLabelSl prefLabelEn scopeNote definition
      broader { uri prefLabel prefLabelSl prefLabelEn }
      narrower { uri prefLabel prefLabelSl prefLabelEn }
      related { uri prefLabel prefLabelSl prefLabelEn }
    }
  }
`;

const GET_NEIGHBORHOOD = gql`
  query GetConceptNeighborhood($uri: String!) {
    conceptNeighborhood(uri: $uri) {
      uri prefLabel prefLabelSl prefLabelEn
    }
    conceptNeighborhoodEdges(uri: $uri) {
      sourceUri targetUri relationType
    }
  }
`;
```

Obe poizvedbi se kličeta z `useQuery` hookom iz `Apollo Client`. `GET_NEIGHBORHOOD` je konfiguriran s `skip: !targetNeighborhoodUri`, kar pomeni, da se izvede samo takrat, ko je `targetNeighborhoodUri` nastavljen - to je mehanizem za leno (lazy) nalaganje soseščine ob kliku na vozlišče v grafu.

5.14 Gradnja grafnih podatkov - useMemo v GraphPage.tsx

Funkcija graphData je zmemoizirana (useMemo) in pretvori Apollo Client podatke v strukturo {nodes, links}, ki jo razume react-force-graph-2d. Ključni del je obravnava hierarhičnih gručk (cluster nodes), ki vizualno grupirajo nadrejene, podrejene in sorodne pojme:

```
const graphData = useMemo<GraphData>(() => {
  if (!data?.concept) return { nodes: [], links: [] };

  // Koreni koncept - pritrjen na koordinati (0, 0)
  rootNode.x = 0; rootNode.y = 0; rootNode.fx = 0; rootNode.fy = 0;

  // Za vsako skupino (broader, narrower, related) ustvari cluster vozlišče
  const addCategoryCluster = (parentUri, kind) => {
    const clusterId = `cluster:${kind}:${parentUri}`;
    nodes.push({ id: clusterId, isCluster: true, clusterKind: kind, ... });
    links.push({ source: parentUri, target: clusterId });
    return clusterId;
  };

  // Ustvari vozlišče za vsak koncept in poveži ga s gruča
  group.items.forEach((item) => {
    getOrAddConceptNode(item, false, clusterSeed);
    links.push({ source: clusterId, target: item.uri });
  });

  return { nodes, links };
}, [data, hiddenCategories, searchLanguage, neighborhoodCache, ...]);
```

Pozicije vozlišč se vzdržujejo v nodePositionCache (Map<string, {x,y}>), ki persists med ponovnimi izrisi grafnih podatkov. Ko se klikne na vozlišče za razširitev grafa, se nova vozlišča dodajo brez premikanja obstoječih.

6. Zagon rešitve

6.1 Predpogoji

Za uspešen lokalni zagon celotnega sklopa (frontend + backend + baza podatkov) so potrebni naslednji predpogoji:

Zahteva	Namen
Node.js 20+	Za pogon frontend razvojnega strežnika (Vite) in npm paketnega upravljalca
npm ali yarn	Upravljalca Node.js paketov za namestitev frontend odvisnosti
Java 21 JDK	Za pogon Spring Boot backend aplikacije (LTS različica)
Gradle 8 (ali gradlew)	Sistem gradnje za backend - priporočeno je uporabiti priložen ./gradlew wrapper
Docker + Docker Compose	Za pogon Neo4j podatkovne baze in opsijsko za celoten sklad
Git	Za kloniranje repozitorijev

6.2 Kloniranje repozitorijev

SGC Navigator je razdeljen v dva ločena repozitorija - frontend in backend. Klonirajte oba:

```
# Klonirajte frontend repozitorij
git clone https://github.com/cobiss-praktikum/frontend.git
cd frontend

# V ločenem terminalu klonirajte backend repozitorij
git clone https://github.com/cobiss-praktikum/backend.git
cd backend
```

6.3 Zagon zalednega sistema (Backend)

6.3.1 Konfiguracija okolja (.env)

V korenu backend repozitorija ustvarite datoteko .env z naslednjimi spremenljivkami:

```
# .env - nikoli ne vključite v verzijsko kontrolo (git ignore!)
NEO4J_URI=bolt://localhost:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=vaše_geslo_neo4j
FRONTEND_URL=url_frontenda
ALTCHA_SECRET=naključen_dolg_skrivni_ključ_za_podpisovanje
```

Opozorilo: Vrednosti `altcha.secret` in `NEO4J_PASSWORD` nikoli ne smejobitih privzetih oziroma kratkih. V produkcijskem okolju so te vrednosti geste, naključno generirane. Nikoli ne objavljajte .env datoteke na Git!

6.3.2 Zagon z Docker Compose (priporočeno za razvoj)

Docker Compose zažene tako Neo4j bazo podatkov kot Spring Boot aplikacijo v ločenih vsebnikih:

```
# V mapi backend repozitorija
docker compose -f docker-compose.dev.yml up --build

# Neo4j browser dostopen na: http://localhost:7474
# Spring Boot API dostopen na: http://localhost:8080
# GraphQL explorer dostopen na: http://localhost:8080/graphql
# Swagger UI dostopen na: http://localhost:8080/swagger-ui
```

6.3.3 Zagon brez Dockerja

Če imate lokalno nameščen Neo4j, lahko zaženete samo Spring Boot:

```
# Prepričajte se, da je Neo4j že zagnan na bolt://localhost:7687

# Zagon Spring Boot
./gradlew bootRun

# Zagon testov (zahteva delujočo Neo4j povezavo)
./gradlew test

# Gradnja JAR datoteke
./gradlew build
```

6.4 Zagon čelnega sistema (Frontend)

6.4.1 Konfiguracija okolja (.env)

V korenu frontend repozitorija ustvarite datoteko .env:

```
# .env za razvoj (Vite proxy posreduje /graphql in /api na ta URL)
VITE_API_URL=http://localhost:8080

# Groq API ključ za AI funkcionalnosti
VITE_GROQ_KEY=
```

6.4.2 Namestitve odvisnosti in zagon

```
# V mapi frontend repozitorija
npm install

# Zagon razvojnega strežnika (HMR na http://localhost:5173)
npm run dev

# Linting (ESLint)
npm run lint

# Testiranje (Vitest)
npm run test

# Produkcijska gradnja v /dist
npm run build

# Predogled produkcijske gradnje
npm run preview
```

6.5 Zagon celotnega sklopa z Docker Compose

Za hkratni zagon frontenda, backenda in baze podatkov v Docker vsebnikih sta na voljo Dockerfile in docker-compose.yml v obeh repozitorijih. Skupen zagon iz korenskega direktorija:

```
# V docker-compose.yml definirani so services: frontend, backend, neo4j
docker-compose up

# Frontend: http://localhost:5173
# Backend:  http://localhost:8080
# Neo4j:    http://localhost:7474
```

Docker slika za backend je večstopenjska (multi-stage build): prva faza (gradle:8-jdk21) prevede in pakira JAR datoteko, druga faza (eclipse-temurin:21-jre-jammy) pa zažene optimizirani JAR brez Gradle odvisnosti. To znatno zmanjša velikost produkcijske Docker slike.

Operacija	Ukaz
Gradnja Docker slike (backend)	docker build -t cobiss-backend:latest .
Zagon Docker vsebnika (backend)	docker run -p 8080:8080 -e NEO4J_URI=... cobiss-backend:latest
Zagon celotnega sklopa	docker-compose up
Zaustavitev in čiščenje	docker-compose down
Čiščenje in ponovna gradnja	docker-compose up --build --force-recreate

6.6 CI/CD cevovod (GitHub Actions)

Oba repozitorija vsebujeta GitHub Actions delovne tokove, ki se samodejno sprožijo ob vsakem push-u na veje main in develop ali zahtevah za združitev (pull requests). Priporočen vrstni red korakov v cevovodu:

Za frontend:

```
steps:
  - name: Lint
    run: npm run lint
  - name: Run tests
    run: npm run test
  - name: Build
    run: npm run build
```

Za backend:

```
steps:
  - name: Run tests
    run: ./gradlew test
  - name: Build
    run: ./gradlew build
```

Opomba: Backend testi zahtevajo delujočo Neo4j instanco. V CI okolju je treba zagotoviti Neo4j kot service container ali mock/testno instanco.

6.7 Dodajanje novih funkcionalnosti - vzorec razvoja

6.7.1 Dodajanje nove GraphQL poizvedbe (Backend)

Standardni 4-koračni postopek za razširitev backend API-ja:

1. Dodajte novo polje v `schema.graphqls` pod type `Query`.
2. Dodajte `@QueryMapping` metodo v `ConceptGraphQLController.java`.
3. Dodajte poslovno logiko v `ConceptService.java`.
4. Dodajte `@Query` Cypher metodo v `ConceptRepository.java` (ali `Neo4jClient` za kompleksne poizvedbe).

6.7.2 Dodajanje novega REST endpoint-a (Backend)

5. Dodajte metodo v `GatewayController.java` (ali ustvarite nov `@RestController`).
6. Zagotovite, da pot ustreza CORS vzorcu v `WebConfig.java`.
7. Dodajte storitev po potrebi.

7. Odprte pomanjkljivosti, nedoslednosti in napake ob predaji rešitve

7.1 Znale pomanjkljivosti

- **N+1 problem pri field-level resolverjih:** Polja broader, narrower in related na tipu Concept so razrešena lenobno prek @SchemaMapping, kar sproži ločeno Neo4j poizvedbo za vsak starševski objekt. Pri iskalni strani (SearchPage), kjer je vrnjenih 20 rezultatov in vsak vsebuje broader/narrower/related, bi to pomenilo do 60+ ločenih poizvedb. Priporočena rešitev: DataLoader pattern ali batch resolverji.
- **CORS je v privzeti konfiguraciji preveč permisiven:** Če spremenljivka FRONTEND_URL ni nastavljena, WebConfig.java privzame http://localhost:5173, kar v produkciji pomeni napako ali preohlapno varnostno politiko. Treba je zagotoviti, da je FRONTEND_URL vedno eksplicitno nastavljen v produkcijskem okolju.
- **sessionStorage za CAPTCHA stanje:** Stanje captcha_verified se shranjuje v sessionStorage, ki se izbriše ob zaprtju zavihka. To pomeni, da mora uporabnik ob vsaki novi brskalni seji znova rešiti CAPTCHA po 10 zahtevah. Za boljšo UX bi bilo bolje shranjevati žeton s časovnim rokom veljavnosti.
- **Brez paginacije pri iskanju:** searchConcepts vrne do limit zadetkov (privzeto 20) brez možnosti paginacije. Za iskanja z veliko zadetki ni možno pridobiti nadaljnjih rezultatov.
- **Testi zahtevajo živo Neo4j instanco:** Integracijski testi v BackendApplicationTests.java zahtevajo realno Neo4j povezavo. Odsotnost mock/test Neo4j instance otežuje testiranje v CI okolju brez posebne infrastrukture.

7.2 Tehnični dolg

- **apollo-client.ts - neaktivna Auth0 koda:** V datoteki apollo-client.ts je prisotna koda za branje Auth0 access_token iz localStorage. Ker Auth0 ni aktivno integriran v produkcijski sistem, ta koda ne deluje in je tehnični dolg, ki bi ga bilo treba ali dokončati ali odstraniti.
- **Komentarji v CaptchaModal.tsx:** V datoteki so zakomentirane vrstice kode za CORS proxy (PROXY_ENDPOINT). To nakazuje na predhodni razvojni eksperiment, ki bi ga bilo treba počistiti.
- **Trdi kodiranje vrednosti v SearchPage.tsx:** Statistika "700.000+" je trdo kodirana v komponenti, čeprav je dejanska vrednost ~75.000. Treba je uskladiti marketing navedbe z dejanskimi podatki.

8. Nadaljnji razvoj

8.1 Kratkoročne izboljšave (0–3 mesece)

- **Implementacija DataLoader vzorca za N+1 optimizacijo:** Zamenjava @SchemaMapping lenobnih resolverjev z DataLoader batch nalaganjem, ki združi vse zahteve za broader/narrower/related v eno Cypher poizvedbo po seriji zahtev.
- **Paginacija iskalnih rezultatov:** Dodajanje parametrov offset in cursor-based paginacije v searchConcepts poizvedbo, skupaj z UI komponentami "Naloži več" ali "Naslednja stran".
- **Izboljšanje testne pokrivenosti:** Dodajanje @TestcontainersNeo4j anotacije za integracijske teste z začasno Neo4j instanco v Docker vsebniku - brez potrebe po ročni bazi.
- **Čiščenje tehničnega dolga:** Odstranitev zakomentiranih vrstic, Auth0 kode in usklajevanje statističnih vrednosti v SearchPage.tsx.

8.2 Srednjeročne izboljšave (3–9 mesecev)

- **Polnobessedilno iskanje (full-text search) z Neo4j indeksi:** Trenutno iskanje temelji na CONTAINS Cypher operatorju, ki ne more koristiti indeksov. Implementacija Neo4j full-text indeksa (CALL db.index.fulltext.createNodeIndex) bi znatno pospešila iskanje pri velikih naborih podatkov.
- **Asmitativno iskanje s predlogi (Autocomplete):** Dodajanje iskalne vrstice z real-time predlogi zadetkov med tipkanjem, implementirano s debounced GraphQL poizvedbo in spustnim menijem.
- **Izvoz grafa:** Možnost izvoza trenutnega pogleda konceptnega grafa v formatih SVG, PNG ali interaktivnem HTML.
- **Upravljanje s koncepti (CRUD):** Za knjižničarje bi bila koristna možnost urejanja, dodajanja in brisanja konceptov in relacij prek zaščitenega administratorskega vmesnika.
- **Napredna vizualizacija s 3D grafom:** Nadgradnja 2D gravitacijskega grafa na 3D vizualizacijo z react-force-graph-3d za bolj intuitivno navigacijo v gostih grafih.

8.3 Dolgoročne izboljšave (9+ mesecev)

- **Semantično iskanje z vektorskim vdeljevanjem (embeddings):** Integracija strojno-učnih modelov za vdeljevanje (npr. SBERT za slovenščino) in implementacija vektorskega iskanja v Neo4j prek GDS (Graph Data Science) knjižnice. Omogočilo bi iskanje po semantičnem pomenu namesto besednega ujemanja.
- **Integracija z BIBLOS / COBISS Plus:** Neposredna integracija iskalnega vmesnika SGC Navigator v COBISS Plus, ki bi knjižničarjem ob katalogizaciji prikazoval vizualni navigator geslovnika.
- **Skupnostne anotacije:** Dodajanje možnosti, da certificirani knjižničarji anotirajo, komentirajo in predlagajo spremembe konceptov prek kontroliranega uredniških delovnega toka.
- **Večjezična razširitev:** Podpora za dodatne jezike (hrvaščina, srbsščina) glede na obstoječe podatke v COBISS regionalnih sistemih.
- **Mobilna aplikacija:** Razvoj React Native ali progresivne spletne aplikacije (PWA) za mobilne naprave z optimiziranim touch vmesnikom za navigacijo po grafu.

8.4 Predlog za prevzem rešitve

Za uspešen prevzem in nadaljnji razvoj rešitve priporočamo naslednje akcije v prvih tednih po predaji:

8. Pregled in uskladitev CORS nastavitve (FRONTEND_URL) v produkcijskem okolju.
9. Nastavitev GitHub Actions skrivnosti (secrets) za CI/CD cevovod.

10. Zamenjava `altcha.secret` z varnim naključnim ključem v produkcijskem `application.properties`.
11. Pregled in sprejemanje odločitve o Auth0 integraciji ali odstranitev mrtve kode.
12. Implementacija Neo4j full-text indeksov za izboljšano iskalno zmogljivost.
13. Dodajanje `@TestcontainersNeo4j` za zanesljivo CI testiranje.

